Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

DevOps Foundation Training

TGS Ref No: TGS-2024049184

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 4.0

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
3.0	Legacy release	Legacy trainer slide release retained as source evidence.	Dr. Alfred Ang
4.0	20 August 2026	Corrected Problem Management TSC mapping; rebuilt eight DevOps domains; added detailed procedures for eight R labs, current DORA context and aligned WA plus Case Study preparation.	Dr. Alfred Ang

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

TABLE OF CONTENTS

How to Use This Guide **4**
Course and WSQ Alignment **4**
Learning Outcomes 4
Knowledge and Ability Statements 4
Assessment Design 5
Source Register and Content Boundary **5**
Environment Setup for All R Labs **5**
DevOps Foundation Knowledge **6**
Topic 01 — Exploring DevOps 6
Topic 02 — Core DevOps Principles 7
Topic 03 — Key DevOps Practices 9
Topic 04 — Business and Technology Frameworks 10
Topic 05 — Culture, Behaviours and Operating Models 11
Topic 06 — Automation and DevOps Toolchains 13
Topic 07 — Measurement, Metrics and Reporting 14
Topic 08 — Sharing, Shadowing and Evolving 15
Detailed R Lab Guides **17**
Lab 01 — Build an Incident-to-Problem Lifecycle Baseline in R 18
Lab 02 — Detect Actionable Service Signals in R 20
Lab 03 — Investigate Recurring Failure Causes with Pareto Analysis 22
Lab 04 — Prioritise Problems by Severity, Frequency and Impact 25
Lab 05 — Diagnose Pipeline Flow and Cultural Debt 27
Lab 06 — Balance Reliability, Security and Delivery Guardrails 29
Lab 07 — Build a DORA and Problem-Management Evidence Dashboard 31
Lab 08 — Capstone — Produce a Verified DevOps Problem Recommendation 33
Assessment Readiness **35**
Glossary **35**

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

How to Use This Guide

This guide places detailed, numbered procedures in the Learner Guide and lab folders. The PowerPoint is intentionally concept-led: use it to understand relationships and decisions, then use this guide to perform and verify each R lab.

- Read the topic explanation before opening its lab folder.
- Work from inside the named Lab-XX folder so all relative data paths resolve.
- Retain the generated CSV, PNG and Markdown evidence plus the completed checklist.
- Use only the supplied synthetic data. Do not use live customer, credential or production telemetry data.
- When a script stops on a validation error, investigate the input and rule; do not remove the check merely to obtain a PASS marker.

Course and WSQ Alignment

Field	Approved value
Course title	DevOps Foundation Training
Course code	TGS-2024049184
TSC	Problem Management (ICT-OUS-4011-1.1)
Duration	16 hours: 14 hours classroom delivery + 2 hours assessment
Assessment	Written Assessment (WA-SAQ), 60 minutes + Case Study (CS), 60 minutes

Learning Outcomes

- LO1: Manage the lifecycle of problems using DevOps principles and technologies.
- LO2: Perform investigations and develop DevOps methods for problem analysis.
- LO3: Recommend DevOps solutions and monitor documentation for problem solving.

Knowledge and Ability Statements

Code	Statement
K1	Principles of problem management throughout its lifecycle
K2	Relevant tools, processes and technologies for problem identification, investigation, analysis and resolution
K3	Problem investigation and diagnosis techniques and methodologies
K4	Problem prioritisation and sizing techniques, methodologies and parameters
K5	Best practices and industry standards in problem-management documentation
A1	Manage the lifecycle of a wide range of problems
A2	Introduce technologies and processes for automated detection of incidents or problems
A3	Investigate and deeply analyse a problem to understand its root causes
A4	Develop guidelines for prioritising and categorising problems by severity, frequency and implications
A5	Recommend solutions that address root causes

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

	and minimise recurrence
A6	Monitor documentation and tracking of problems encountered and resolved

Assessment Design

- Written Assessment (WA-SAQ) — five open-ended questions, 60 minutes, open book.
- Case Study (CS) — three open-ended tasks, 60 minutes, open book.
- Assessment design mirrors the current Drive-held v2 instruments: 5 WA questions (K1–K5) and 3 CS tasks (A1–A6).

Note: The current Drive-held v2 papers establish the controlled structure: five WA questions covering K1–K5 and three Case Study tasks covering A1–A6. This release revises content, not instrument type, count, timing or mapping.

Source Register and Content Boundary

Authority	Source	Use in v4.0
Content source	Legacy v3 PowerPoint	Preserved useful coverage across all eight DevOps Foundation domains; legacy design was not reused.
WSQ authority	CP_TIPL_devops_v2.docx and TSC Mapping-TI-devops.docx	Course duration, learning outcomes, K1–K5, A1–A6, 14-hour delivery and WA+CS assessment design.
Certification blueprint	DevOps Institute DOFD v3.4 English Study Guide	Eight modules, vocabulary, course objectives and 40-question topic weighting.
Current certification page	PeopleCert DevOps Foundation	Current high-level positioning and evolving practices including platform engineering and generative AI.
Current performance research	DORA software delivery performance metrics	Current five-metric throughput and instability model; historical four-key terminology is explicitly distinguished.
Secondary study guidance	Coursera, Passnova, Test-King and SKILup pages supplied by the user	Used for study-planning emphasis and hands-on reinforcement only; no exam dumps or proprietary questions reproduced.

Note: The legacy deck is a content source, not a design template. No sample examination question from the DevOps Institute guide or any secondary preparation site is reproduced in this courseware.

Environment Setup for All R Labs

1. Install R 4.x from the official R project distribution for your operating system.
2. Open a new terminal and run `Rscript --version`; resolve PATH issues before class.
3. Download the complete `labs/` folder from LMS-TMS or GitHub while preserving folder names.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

4. Open a terminal in the specific Lab-XX folder—not at the repository root.
5. Run the lab's supplied R script. Every script uses base R only and requires no external package.
6. Confirm the expected `LAB NN PASS` marker, inspect each output and complete `CHECKLIST.md`.

```
Rscript --version

cd "labs/Lab-01-incident-lifecycle-baseline"

Rscript incident_lifecycle_baseline.R
```

DevOps Foundation Knowledge

Topic 01 — Exploring DevOps

Alignment: LO1 · K1 · A1 · DevOps Foundation blueprint emphasis: 5 / 40 exam questions.

Purpose · business value · problem lifecycle

Key ideas

Idea	Explanation
DevOps is a sociotechnical system	People, process and technology optimise one value stream; a tools-only programme is not DevOps.
Shared ownership replaces hand-offs	Development, operations, quality and security own flow and reliability together.
Small batches expose problems early	Frequent, reversible changes shorten feedback and reduce the blast radius of failure.
Problems become learning assets	Detection, investigation, resolution and prevention are connected through explicit records and feedback.

DevOps: What It Is — and Is Not

Dimension	DevOps	Common misconception
Identity	A culture and operating model	A new job title or team
Goal	Fast, safe flow of customer value	Maximum deployment volume
Scope	Idea through operate and learn	Only CI/CD automation
Control	Evidence, guardrails and feedback	Removing governance

Note: Use this distinction when diagnosing a transformation that has bought tools but retained siloed incentives.

The Problem Lifecycle in a DevOps System

7. Detect: Telemetry reveals a signal
8. Log: Create one problem record
9. Investigate: Evidence before opinion
10. Resolve: Treat cause and impact
11. Learn: Verify and prevent recurrence

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: LO1 CONNECTION — A1 is demonstrated by managing the whole lifecycle, not merely closing an incident.

Why Waiting Dominates Lead Time

Category	Active work (hours)	Queue / wait (hours)
Design	4	8
Build	6	12
Review	2	18
Test	3	10
Release	1	20

Note: The largest improvement opportunity is often the queue between teams, not faster typing inside one stage.

DevOps Business Outcomes

Concept	Explanation
Speed	Shorter lead time turns feedback into value sooner.
Stability	Small changes and automation make releases easier to diagnose and reverse.
Quality	Testing, observability and shared ownership prevent defects escaping unnoticed.
Learning	Blameless review converts failure into changes to the system of work.

Stakeholders in One Value Stream

Product: Clarifies value and outcome; Prioritises by customer impact.

Engineering: Builds in small testable increments; Owns operability with operations.

Operations & Security: Designs reliability guardrails; Feeds production learning upstream.

The Core Shift

Optimise the whole system, not a local silo.

Note: A faster development team does not create faster value when review, release or recovery remains constrained.

Note: Apply this topic in Lab 01: Build an Incident-to-Problem Lifecycle Baseline in R.

Topic 02 — Core DevOps Principles

Alignment: LO1 · K2 · A2 · DevOps Foundation blueprint emphasis: 4 / 40 exam questions.

Three Ways · constraints · learning · automated feedback

Key ideas

Idea	Explanation
First Way — flow	Visualise work, reduce batch size and remove the system constraint.
Second Way — feedback	Build fast signals from production and customers back to the people changing the system.
Third Way — learning	Create safety for experimentation, repetition and

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

	improvement.
Automation supports judgement	Automate repeatable checks; keep people accountable for thresholds, risk and learning.

The Three Ways

12. Flow: Left to right value
13. Feedback: Right to left learning
14. Learning: Experiment and improve

Note: SYSTEM VIEW — All three reinforce each other: flow without feedback accelerates mistakes; feedback without learning repeats them.

Traditional Batch vs Continuous Flow

Design choice	Traditional batch	Continuous flow
Change size	Large release package	Small independently releasable change
Integration	Late	At least daily
Feedback	After release window	At every pipeline stage
Recovery	Complex rollback	Fast revert or progressive rollout

Theory of Constraints Improvement Cycle

15. Identify: Find the current constraint
16. Exploit: Protect its useful capacity
17. Subordinate: Align other work to it
18. Elevate: Increase capacity or redesign
19. Repeat: Find the next constraint

Note: CAUTION — Optimising a non-constraint creates local efficiency but not better end-to-end flow.

Fast Feedback Signals

Concept	Explanation
Commit	Static checks and unit tests respond in minutes.
Pipeline	Build, integration and security checks create evidence.
Production	Logs, metrics, traces and user journeys show real behaviour.
Customer	Outcome and support feedback closes the value loop.

Fail-Safe vs Safe-to-Fail

Question	Fail-safe mindset	Safe-to-fail mindset
Assumption	Failure can be designed out	Failure will occur
Response	Add approval and prediction	Limit impact and recover fast
Practices	Large pre-release gate	Feature flags, canary, rollback
Learning	Find who deviated	Improve conditions and controls

Note: Safety culture does not mean accepting reckless change; it means making risk visible, bounded and learnable.

Automation Principle

Automate the feedback, not the thinking.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: A threshold can detect a symptom; people still determine relevance, root cause, priority and the systemic response.

Note: Apply this topic in Lab 02: Detect Actionable Service Signals in R.

Topic 03 — Key DevOps Practices

Alignment: LO1 · K2 · A2 · DevOps Foundation blueprint emphasis: 7 / 40 exam questions.

CI/CD · testing · SRE · resilience · DevSecOps · ChatOps · Kanban

Key ideas

Idea	Explanation
Continuous integration	Frequent integration into a shared mainline, verified by an automated build and tests.
Continuous delivery	Every accepted change remains deployable; a business decision may still trigger production release.
Continuous deployment	Every accepted change automatically reaches production.
Operability by design	SRE, observability, resilience and security are part of the product, not downstream inspections.

A Modern Delivery Pipeline

20. Commit: Versioned change
21. Build: Reproducible artifact
22. Test: Quality and security
23. Release: Policy and evidence
24. Deploy: Progressive and reversible
25. Observe: Service and user signals

Note: CONTROL POINT — Each stage must produce evidence that the next stage can trust.

CI, Delivery and Deployment

Practice	End state	Human production decision?
Continuous integration	Tested integrated code	Not applicable
Continuous delivery	Always releasable artifact	Usually yes
Continuous deployment	Accepted change in production	No manual release gate

Progressive Delivery Reduces Blast Radius

Category	Users initially exposed (%)	Rollback effort index
Big-bang	100	10
Blue-green	50	5
Canary 10%	10	2
Feature flag	5	1

Note: Progressive strategies reduce initial exposure and make rollback cheaper; they require strong observability.

Reliability Engineering in Practice

SRE: Service-level objectives; Error budgets balance reliability and change.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Resilience: Design for graceful degradation; Test recovery assumptions.

Observability: Logs, metrics and traces; Ask new questions without new code.

DevSecOps: Security as Continuous Evidence

Concept	Explanation
Shift left	Threat modelling, dependencies and code scanning before merge.
Protect pipeline	Signed artifacts, least privilege and isolated secrets.
Shift right	Runtime detection, attack simulation and rapid containment.
Shared accountability	Security supplies guardrails; product teams own secure outcomes.

ChatOps and Kanban

Practice	Makes visible	Problem-management value
ChatOps	Conversation, actions and evidence	One shared incident narrative
Kanban	Work, policy and blocked state	Limits work in progress and exposes ageing
Together	Flow plus collaboration	Faster coordinated diagnosis and resolution

Note: Apply this topic in Lab 03: Investigate Recurring Failure Causes with Pareto Analysis.

Topic 04 — Business and Technology Frameworks

Alignment: LO2 · K3 · A3 · DevOps Foundation blueprint emphasis: 7 / 40 exam questions.

Agile · Lean · ITSM · safety culture · learning organisation

Key ideas

Idea	Explanation
Agile improves discovery	Short iterations and customer feedback refine what to build.
Lean improves flow	Remove delay, overproduction, hand-offs and rework from the value stream.
ITSM provides service discipline	Incident, problem and change practices create ownership and traceability.
Safety enables truthful evidence	People report weak signals and mistakes when the organisation learns rather than blames.

How the Frameworks Complement DevOps

Framework	Primary lens	Contribution
Agile	Adaptive product delivery	Small increments and customer feedback
Lean	End-to-end value flow	Waste reduction and constraint focus
ITSM	Reliable service management	Ownership, records and controlled change
DevOps	Sociotechnical delivery system	Integrates culture, flow, automation and learning

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Evidence-Led Investigation

26. Frame: Define impact and scope
27. Collect: Timeline, telemetry, changes
28. Hypothesise: Candidate causes
29. Test: Falsify with evidence
30. Conclude: Cause and contributing conditions

Note: A3 STANDARD — Separate observed fact, inference and decision in the problem record.

Root-Cause Techniques — Choose by Evidence

Concept	Explanation
5 Whys	Fast exploration of a causal chain; stop when evidence stops.
Fishbone	Organise candidate causes across people, process, technology and environment.
Timeline	Reconstruct what changed and when; useful for complex incidents.
Pareto	Identify the few recurring categories that drive most impact.

Flow Efficiency Example

Category	Touch time (%)	Waiting time (%)
Feature A	22	78
Feature B	18	82
Feature C	31	69
Hotfix	45	55

Note: Most elapsed time is waiting. A Lean investigation asks why work is queued, blocked or handed off.

Incident vs Problem vs Change

Record	Question	Done when
Incident	How do we restore service?	Impact is controlled
Problem	Why did it happen and recur?	Cause and prevention are managed
Change	How do we alter the service safely?	Evidence shows intended outcome

Just Culture

Hold people accountable for learning, not for being human.

Note: Examine choices in context, improve the system, and distinguish honest error from reckless behaviour.

Note: Apply this topic in Lab 04: Prioritise Problems by Severity, Frequency and Impact.

Topic 05 — Culture, Behaviours and Operating Models

Alignment: LO2 · K4 · A4 · DevOps Foundation blueprint emphasis: 6 / 40 exam questions.

CALMS · cultural debt · Westrum · maturity · team design

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Key ideas

Idea	Explanation
Culture is observable behaviour	How information flows, risk is escalated and decisions are made matters more than slogans.
Cultural debt compounds	Workarounds, fear and unclear ownership make each future change slower and riskier.
Priority policy must be explicit	Severity, frequency, impact and urgency should be scored consistently.
Maturity is contextual	Use models to guide conversation and experiments, never as a vanity ranking.

CALMS as a Transformation Lens

Culture: Shared ownership; Psychological safety and trust.

Automation & Lean: Repeatable feedback; Small batches and reduced waste.

Measurement & Sharing: Outcome evidence; Learning across boundaries.

Westrum Organisational Cultures

Behaviour	Pathological	Bureaucratic	Generative
Information	Hidden	Channels enforced	Actively sought
Failure	Scapegoat	Local justice	Inquiry and learning
Novelty	Crushed	Creates problems	Implemented
Responsibility	Avoided	Compartmentalised	Shared

From Signal to Priority

31. Severity: How bad is the impact?
32. Frequency: How often does it recur?
33. Reach: Who and what is affected?
34. Risk: What could happen next?
35. Priority: Sequence with capacity and evidence

Note: A4 OUTPUT — Publish the scoring rule so two teams reach comparable decisions from the same evidence.

Priority Is Not Severity Alone

Category	Severity	Frequency	Impact
Login outage	5	2	5
Slow report	2	5	3
Rare data defect	5	1	5
Alert noise	1	5	2

Note: A severe rare issue and a lower-severity recurring issue can both deserve attention for different reasons.

Team Operating Models

Concept	Explanation
Product team	Owns customer outcome and service operation end to end.
Platform team	Provides a paved road that reduces cognitive load.
Enabling team	Builds capability temporarily, then moves on.
Complicated subsystem	Owns specialist components requiring deep expertise.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Maturity Principle

Improve the next constraint; do not perform maturity theatre.

Note: A useful maturity model reveals the next experiment and evidence of progress—not a trophy score.

Note: Apply this topic in Lab 05: Diagnose Pipeline Flow and Cultural Debt.

Topic 06 — Automation and DevOps Toolchains

Alignment: LO2 · K4 · A4 · DevOps Foundation blueprint emphasis: 5 / 40 exam questions.

CI/CD · cloud · containers · microservices · IaC · AIOps · platform engineering

Key ideas

Idea	Explanation
Design the value stream first	A toolchain should serve a clear flow and evidence model, not automate a broken process.
Everything important is versioned	Code, configuration, infrastructure, policy and documentation become reviewable and reproducible.
Loose coupling enables flow	Services and teams can test, deploy, scale and fail independently.
Platforms create safe self-service	A paved road standardises guardrails while preserving team autonomy.

Reference DevOps Toolchain

36. Plan: Work and value
37. Code: Version control
38. Build: Artifact
39. Test: Quality and security
40. Deploy: Progressive change
41. Operate: Observe and learn

Note: ARCHITECTURE RULE — Integrate evidence and ownership between tools; avoid a collection of disconnected products.

Architecture Choices

Choice	Benefit	Cost / risk
Cloud	Elastic self-service resources	Cost, identity and provider governance
Containers	Portable immutable packaging	Image and runtime security
Microservices	Independent change and scale	Distributed-system complexity
IaC	Repeatable reviewed infrastructure	State, drift and secret handling

Automation Selection Test

Concept	Explanation
Repeatable	The task occurs often enough to justify automation.
Deterministic	Inputs, rules and acceptable outputs can be specified.
Observable	Success and failure produce usable evidence.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Recoverable	A failed automation can stop, retry or roll back safely.

Automation Economics

Category	Setup effort index	Effort per change	Consistency
Manual release	1	10	2
Scripted	3	6	5
Pipeline	7	2	8
Platform paved road	10	1	10

Note: More capable automation costs more initially but reduces effort and variation across repeated changes.

AIOps and Generative AI — Useful Roles

Detect: Cluster related signals; Identify unusual patterns.

Assist: Summarise evidence; Suggest hypotheses for human review.

Govern: Track confidence and provenance; Require approval for high-impact action.

Toolchain vs Platform

Dimension	Toolchain	Internal platform
Focus	Connected lifecycle tools	Self-service developer experience
User action	Navigate each tool	Use a paved workflow or portal
Governance	Policies per tool	Guardrails embedded in the path
Success	Integrated evidence	Reduced cognitive load and lead time

Note: Apply this topic in Lab 06: Balance Reliability, Security and Delivery Guardrails.

Topic 07 — Measurement, Metrics and Reporting

Alignment: LO3 · K5 · A6 · DevOps Foundation blueprint emphasis: 2 / 40 exam questions.

DORA · reliability · flow · business value · problem records

Key ideas

Idea	Explanation
Measure outcomes and system health	A balanced view covers throughput, instability, reliability, quality and business value.
Trends beat targets	Use metrics to learn over time, not to rank individuals or encourage gaming.
Context matters	Compare one service with itself and record changes in architecture, demand and risk.
Documentation is operational telemetry	Problem records show whether causes, actions and verification are actually closed.

Current DORA Software Delivery Metrics

Throughput: Change lead time; Deployment frequency; Failed deployment recovery time.

Instability: Change fail rate; Deployment rework rate.

Context: Use per service; Pair with reliability and business outcomes.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Baseline and Target — Example Service

Category	Baseline	90-day target
Deployments / week	2	5
Lead time (hours)	48	20
Change fail rate (%)	24	15
Recovery time (hours)	8	3
Rework rate (%)	18	10

Note: The chart mixes units deliberately for a leadership conversation; operational dashboards should separate scales and retain definitions.

Metric, KPI, SLI, SLO and SLA

Term	Meaning	Example
Metric	Any quantitative measure	Request latency
KPI	Measure tied to a goal	Checkout completion rate
SLI	Observed service measure	99.93% successful requests
SLO	Internal reliability target	99.9% success over 30 days
SLA	External commitment	Credit if monthly availability falls below contract

Measurement-to-Improvement Loop

42. Define: Question and measure
43. Collect: Trusted data
44. Interpret: Context and trend
45. Decide: Experiment or action
46. Verify: Outcome and learning

Note: ANTI-PATTERN — A dashboard without a decision cadence is decoration.

Problem Record — Minimum Evidence

Concept	Explanation
Identity	Unique ID, service, owner, status and timestamps.
Impact	Users, duration, severity and business consequence.
Analysis	Timeline, evidence, cause and contributing conditions.
Action	Recommendation, accountable owner, due date and verification.

Healthy vs Harmful Metrics

Healthy use	Harmful use
Team learns from trends	Individuals are ranked by output
Definitions are explicit	Numbers are compared across unlike services
Qualitative context is retained	One target becomes the whole goal
Measures drive experiments	Measures drive blame or gaming

Note: Apply this topic in Lab 07: Build a DORA and Problem-Management Evidence Dashboard.

Topic 08 — Sharing, Shadowing and Evolving

Alignment: LO3 · K5 · A5 · DevOps Foundation blueprint emphasis: 4 / 40 exam questions.

Enterprise adoption · leadership · roles · communities · critical success factors

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Key ideas

Idea	Explanation
Adoption is an evolving product	Start with a value stream, learn, improve the platform and expand through evidence.
Leaders design conditions	Align incentives, fund products, remove systemic constraints and model learning behaviour.
Sharing scales learning	Communities, pairing, shadowing and reusable patterns prevent isolated success.
Recommendations need verification	State cause, action, owner, due date, risk and the signal that proves recurrence fell.

A Safe Enterprise Adoption Path

47. Select: One value stream
48. Baseline: Flow, quality and reliability
49. Experiment: Small bounded change
50. Platformise: Reuse a proven path
51. Scale: Communities and evidence

Note: CRITICAL SUCCESS FACTOR — Scale learning and enabling capabilities before mandating standard tools everywhere.

Leadership Responsibilities

Direction: Connect DevOps to business outcomes; Make trade-offs explicit.

System: Fund products and platforms; Remove conflicting incentives.

Learning: Model blameless inquiry; Protect time for improvement.

Ways to Spread Capability

Mechanism	Best use	Watch for
Community of practice	Shared patterns and learning	Meeting without ownership
Pairing / shadowing	Tacit skill transfer	Dependency on one expert
Enablement team	Accelerated adoption	Permanent ticket queue
Internal platform	Safe repeatable self-service	Platform built without user research

Adoption Evidence Over Four Quarters

Category	Teams on paved road	Median lead-time improvement (%)	Repeat incidents reduced (%)
Q1	1	8	4
Q2	3	18	12
Q3	7	31	25
Q4	12	42	38

Note: Scale is credible when adoption and outcome evidence rise together—not when tool licences alone increase.

Critical Success Factors

Concept	Explanation
Outcome	A visible business and service problem worth solving.
Ownership	Cross-functional team owns build, run and learn.
Architecture	Small independently changeable components where practical.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Feedback	Trusted telemetry, customer signals and regular review.
Platform	Low-friction compliant paths for common work.
Leadership	Funding, incentives and behaviour support long-term change.

From Root Cause to Verified Recommendation

52. State cause: Evidence and conditions
53. Design control: Prevent, detect or recover
54. Assess trade-off: Value, cost and risk
55. Assign: Owner and due date
56. Verify: Metric and review date

Note: A5 OUTPUT — A recommendation is incomplete until its success criterion and follow-up are recorded.

Note: Apply this topic in Lab 08: Capstone — Produce a Verified DevOps Problem Recommendation.

Detailed R Lab Guides

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 01 — Build an Incident-to-Problem Lifecycle Baseline in R

Field	Lab specification
Topic	Topic 01
Objective	Apply K1 and A1 by managing a supplied incident dataset through detection, logging, investigation, resolution and learning states.
Scenario	Northstar Retail has recurring checkout incidents but no shared view of where work stalls. You are asked to build a reproducible lifecycle baseline before proposing automation.
Output	A clean lifecycle_summary.csv, ageing_problems.csv and lifecycle_status.png backed by a repeatable R script.
Tools	R 4.x · base R · CSV · PNG evidence

Before you run the script

- Open a terminal in this lab's folder and confirm `Rscript --version`.
- Inspect every source CSV and note its row count and key identifiers.
- Preserve the supplied source data; make a copy for sensitivity experiments.
- Create no production connection and add no real personal, credential or customer information.

Step-by-step procedure

1. Open a terminal in the Lab-01 folder and confirm R is available.

```
Rscript --version
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

2. Inspect data/incidents.csv and identify the required identity, impact, ownership and timestamp fields.

```
Rscript -e 'd <- read.csv("data/incidents.csv"); str(d); print(d)'
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

3. Run the supplied analysis script; it validates statuses and dates before calculating hours to restore and days open.

```
Rscript incident_lifecycle.R
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

4. Review output/lifecycle_summary.csv and explain where the lifecycle is accumulating delay.

```
Rscript -e 'print(read.csv("output/lifecycle_summary.csv"))'
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

5. Review output/ageing_problems.csv and choose one record for further investigation.

```
Rscript -e 'print(read.csv("output/ageing_problems.csv"))'
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

6. Open output/lifecycle_status.png and record two evidence-based observations in CHECKLIST.md.

```
open output/lifecycle_status.png
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

Evidence to retain

- output/lifecycle_summary.csv
- output/ageing_problems.csv
- output/lifecycle_status.png
- completed CHECKLIST.md

Acceptance criteria

- No invalid status or negative duration
- Status total equals eight source records
- At least one ageing problem is identified
- Two observations cite values from the outputs

Final verification

The script exits with 'LAB 01 PASS'; all eight records are represented; open problems are ranked by ageing days; status totals reconcile to the source rows.

Troubleshooting

Symptom	Check and recovery
Rscript not found	Confirm R is installed, reopen the terminal and verify PATH with `Rscript --version`.
Cannot open data file	Confirm the terminal is inside this Lab-XX folder and the `data/` directory retains its name.
Validation stop	Read the named rule, inspect the source row and correct only the copied experiment data or your working assumption.
Output differs	Reconcile IDs, row counts, denominators and units before changing the formula.
Chart does not open	Open the PNG directly from `output/`; the evidence CSV remains authoritative.

Note: The same detailed guide and checklist are also packaged in `labs/Lab-01-incident-lifecycle-baseline/` as Markdown and PDF.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 02 — Detect Actionable Service Signals in R

Field	Lab specification
Topic	Topic 02
Objective	Apply K2 and A2 by designing an automated detection rule that produces specific, severity-ranked problem signals without excessive noise.
Scenario	Checkout latency has intermittent spikes. The operations channel receives too many generic alerts, so the team needs an evidence-based detection rule and clear routing fields.
Output	A detected_alerts.csv with severity, evidence and action plus a telemetry_detection.png chart.
Tools	R 4.x · base R · telemetry · automated detection

Before you run the script

- Open a terminal in this lab's folder and confirm `Rscript --version`.
- Inspect every source CSV and note its row count and key identifiers.
- Preserve the supplied source data; make a copy for sensitivity experiments.
- Create no production connection and add no real personal, credential or customer information.

Step-by-step procedure

1. Review the telemetry sample and identify the service-level indicators represented by each column.

```
Rscript -e 'd <- read.csv("data/telemetry.csv"); summary(d)'
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

2. Run automated_detection.R to calculate a rolling baseline and flag only sustained abnormal windows.

```
Rscript automated_detection.R
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

3. Inspect detected_alerts.csv; confirm every alert names the service, signal, severity, evidence and immediate action.

```
Rscript -e 'print(read.csv("output/detected_alerts.csv"))'
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

4. Open the detection chart and compare the threshold with the observed latency series.

```
open output/telemetry_detection.png
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

5. Change min_consecutive from 2 to 1, rerun, and compare the alert count to demonstrate alert-fatigue risk; then restore it to 2.

```
Rscript automated_detection.R
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

6. Complete CHECKLIST.md with the chosen rule, trade-off and escalation owner.

```
open CHECKLIST.md
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

Evidence to retain

- output/detected_alerts.csv
- output/telemetry_detection.png
- completed CHECKLIST.md

Acceptance criteria

- Two sustained alert windows
- No alert for isolated row 7 spike
- Every alert has severity, owner and action
- Learner explains sensitivity versus noise

Final verification

The final rule emits exactly two sustained alert windows, no single-point spike alert, and every alert has a non-empty owner and action.

Troubleshooting

Symptom	Check and recovery
Rscript not found	Confirm R is installed, reopen the terminal and verify PATH with `Rscript --version`.
Cannot open data file	Confirm the terminal is inside this Lab-XX folder and the `data/` directory retains its name.
Validation stop	Read the named rule, inspect the source row and correct only the copied experiment data or your working assumption.
Output differs	Reconcile IDs, row counts, denominators and units before changing the formula.
Chart does not open	Open the PNG directly from `output/`; the evidence CSV remains authoritative.

Note: The same detailed guide and checklist are also packaged in `labs/Lab-02-automated-signal-detection/` as Markdown and PDF.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 03 — Investigate Recurring Failure Causes with Pareto Analysis

Field	Lab specification
Topic	Topic 03
Objective	Apply K2 and A2 while preparing evidence for A3: distinguish symptoms from cause candidates and identify recurring categories worth deeper investigation.
Scenario	Pipeline failures are discussed anecdotally. The team needs a quantified Pareto view before choosing a root-cause investigation target.
Output	A cause_pareto.csv, pareto_failures.png and five_whys_template.csv.
Tools	R 4.x · base R · Pareto · 5 Whys

Before you run the script

- Open a terminal in this lab's folder and confirm `Rscript --version`.
- Inspect every source CSV and note its row count and key identifiers.
- Preserve the supplied source data; make a copy for sensitivity experiments.
- Create no production connection and add no real personal, credential or customer information.

Step-by-step procedure

1. Inspect failure_events.csv and distinguish observed category, symptom and lost-time fields.

```
Rscript -e 'd <- read.csv("data/failure_events.csv"); str(d)'
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

2. Run root_cause_analysis.R to aggregate categories and calculate cumulative contribution.

```
Rscript root_cause_analysis.R
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

3. Review cause_pareto.csv and select the smallest set of categories accounting for at least 80% of lost minutes.

```
Rscript -e 'print(read.csv("output/cause_pareto.csv"))'
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

4. Open the Pareto chart and verify the bar and cumulative-line relationship.

```
open output/pareto_failures.png
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

5. Complete five_whys_template.csv for the leading category using facts from the dataset; mark unsupported statements as hypotheses.

```
open output/five_whys_template.csv
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

6. Record the selected investigation target and required next evidence in CHECKLIST.md.

```
open CHECKLIST.md
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

Evidence to retain

- output/cause_pareto.csv
- output/pareto_failures.png
- completed output/five_whys_template.csv
- completed CHECKLIST.md

Acceptance criteria

- Cumulative percentage ends at 100
- Leading cause is evidence-based
- 5 Whys separates facts and hypotheses
- Next evidence is specified

Final verification

LAB 03 PASS is printed; cumulative percentage ends at 100%; the leading category is supported by both count and lost-minute evidence.

Troubleshooting

Symptom	Check and recovery
Rscript not found	Confirm R is installed, reopen the terminal and verify PATH with `Rscript --version`.
Cannot open data file	Confirm the terminal is inside this Lab-XX folder and the `data/` directory retains its name.
Validation stop	Read the named rule, inspect the source row and correct only the copied experiment data or your working assumption.
Output differs	Reconcile IDs, row counts, denominators and units before changing the formula.
Chart does not open	Open the PNG directly from `output/`; the evidence CSV remains authoritative.

Note: The same detailed guide and checklist are also packaged in `labs/Lab-03-root-cause-pareto/` as Markdown and PDF.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 04 — Prioritise Problems by Severity, Frequency and Impact

Field	Lab specification
Topic	Topic 04
Objective	Apply K3 and A3 by defining evidence, then K4 and A4 by applying a transparent prioritisation rule.
Scenario	Northstar Retail can fund only two problem-remediation initiatives this quarter. Leaders disagree because each team uses a different definition of priority.
Output	A ranked problem_priority.csv and priority_matrix.png with documented scoring assumptions.
Tools	R 4.x · base R · prioritisation · impact analysis

Before you run the script

- Open a terminal in this lab's folder and confirm `Rscript --version`.
- Inspect every source CSV and note its row count and key identifiers.
- Preserve the supplied source data; make a copy for sensitivity experiments.
- Create no production connection and add no real personal, credential or customer information.

Step-by-step procedure

1. Review problems.csv and confirm all scoring fields use the documented 1–5 scale.

```
Rscript -e 'd <- read.csv("data/problems.csv"); summary(d)'
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

2. Run problem_prioritisation.R to calculate impact-weighted priority and rank the backlog.

```
Rscript problem_prioritisation.R
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

3. Inspect the top two problems and explain why they rank above a severe but low-confidence item.

```
Rscript -e 'print(read.csv("output/problem_priority.csv"))'
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

4. Open the severity-frequency matrix and compare visual position with the final score.

```
open output/priority_matrix.png
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

5. Run one sensitivity test by increasing an effort estimate or reducing confidence, then restore the source file.

```
Rscript problem_prioritisation.R
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

6. Document the scoring policy, selected initiatives and trade-off in CHECKLIST.md.

```
open CHECKLIST.md
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

Evidence to retain

- output/problem_priority.csv
- output/priority_matrix.png
- completed CHECKLIST.md

Acceptance criteria

- Formula is explicit
- Unique rank for all problems
- Top two selected with evidence
- Sensitivity test is explained

Final verification

Every score is reproducible from the published formula; ranks are unique; the top two recommendations cite impact, evidence confidence and effort.

Troubleshooting

Symptom	Check and recovery
Rscript not found	Confirm R is installed, reopen the terminal and verify PATH with `Rscript --version`.
Cannot open data file	Confirm the terminal is inside this Lab-XX folder and the `data/` directory retains its name.
Validation stop	Read the named rule, inspect the source row and correct only the copied experiment data or your working assumption.
Output differs	Reconcile IDs, row counts, denominators and units before changing the formula.
Chart does not open	Open the PNG directly from `output/`; the evidence CSV remains authoritative.

Note: The same detailed guide and checklist are also packaged in `labs/Lab-04-problem-prioritisation/` as Markdown and PDF.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 05 — Diagnose Pipeline Flow and Cultural Debt

Field	Lab specification
Topic	Topic 05
Objective	Apply K4 and A4 by combining stage performance with visible queue and rework evidence to recommend a prioritised improvement.
Scenario	Teams blame the test stage for slow delivery, but no one has quantified queue time, rework or manual hand-offs across the pipeline.
Output	A pipeline_stage_summary.csv, pipeline_flow.png and a generative-culture recommendation.
Tools	R 4.x · base R · flow metrics · culture evidence

Before you run the script

- Open a terminal in this lab's folder and confirm `Rscript --version`.
- Inspect every source CSV and note its row count and key identifiers.
- Preserve the supplied source data; make a copy for sensitivity experiments.
- Create no production connection and add no real personal, credential or customer information.

Step-by-step procedure

1. Inspect pipeline_runs.csv and identify active time, wait time, failures and rework.

```
Rscript -e 'd <- read.csv("data/pipeline_runs.csv"); str(d)'
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

2. Run pipeline_reliability.R to calculate stage-level flow and instability indicators.

```
Rscript pipeline_reliability.R
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

3. Review pipeline_stage_summary.csv and identify the current constraint using evidence rather than reputation.

```
Rscript -e 'print(read.csv("output/pipeline_stage_summary.csv"))'
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

4. Open pipeline_flow.png and locate the stage with the largest wait-to-active ratio.

```
open output/pipeline_flow.png
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

5. Translate the finding into one CALMS improvement and one generative-culture behaviour.

```
open CHECKLIST.md
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

6. Record a success metric and 30-day review date for the recommendation.

```
open CHECKLIST.md
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

Evidence to retain

- output/pipeline_stage_summary.csv
- output/pipeline_flow.png
- completed CHECKLIST.md

Acceptance criteria

- Constraint identified from data
- No individual-blame language
- CALMS dimension named
- Metric and review date recorded

Final verification

The identified constraint matches the maximum wait ratio; the recommendation changes a system condition and names an outcome measure.

Troubleshooting

Symptom	Check and recovery
Rscript not found	Confirm R is installed, reopen the terminal and verify PATH with `Rscript --version`.
Cannot open data file	Confirm the terminal is inside this Lab-XX folder and the `data/` directory retains its name.
Validation stop	Read the named rule, inspect the source row and correct only the copied experiment data or your working assumption.
Output differs	Reconcile IDs, row counts, denominators and units before changing the formula.
Chart does not open	Open the PNG directly from `output/`; the evidence CSV remains authoritative.

Note: The same detailed guide and checklist are also packaged in `labs/Lab-05-pipeline-risk-review/` as Markdown and PDF.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 06 — Balance Reliability, Security and Delivery Guardrails

Field	Lab specification
Topic	Topic 06
Objective	Apply K4 and A4 by categorising reliability and security risks, then selecting proportional automated guardrails.
Scenario	A team wants faster deployment, while operations and security want more approval gates. You must use risk and error-budget evidence to design a faster but controlled path.
Output	A guardrail_decisions.csv, error_budget.png and documented trade-off decision.
Tools	R 4.x · base R · SLO · DevSecOps · policy evidence

Before you run the script

- Open a terminal in this lab's folder and confirm `Rscript --version`.
- Inspect every source CSV and note its row count and key identifiers.
- Preserve the supplied source data; make a copy for sensitivity experiments.
- Create no production connection and add no real personal, credential or customer information.

Step-by-step procedure

1. Inspect service_risk.csv and review the SLO, observed reliability and security finding fields.

```
Rscript -e 'print(read.csv("data/service_risk.csv"))'
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

2. Run resilience_security.R to calculate error-budget consumption and security risk score.

```
Rscript resilience_security.R
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

3. Review guardrail_decisions.csv and explain why different services receive warn, approve or block treatment.

```
Rscript -e 'print(read.csv("output/guardrail_decisions.csv"))'
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

4. Open error_budget.png and identify the service consuming more than its available budget.

```
open output/error_budget.png
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

5. Propose one preventive, one detective and one recovery control for the highest-risk service.

```
open CHECKLIST.md
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

6. Record the control owner, evidence source and safe rollback method.

```
open CHECKLIST.md
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

Evidence to retain

- output/guardrail_decisions.csv
- output/error_budget.png
- completed CHECKLIST.md

Acceptance criteria

- Error budget formula correct
- Three distinct proportional policies
- Prevent/detect/recover controls
- Rollback and owner recorded

Final verification

All policies are derived from published thresholds; at least one service is blocked, one warns and one proceeds automatically.

Troubleshooting

Symptom	Check and recovery
Rscript not found	Confirm R is installed, reopen the terminal and verify PATH with `Rscript --version`.
Cannot open data file	Confirm the terminal is inside this Lab-XX folder and the `data/` directory retains its name.
Validation stop	Read the named rule, inspect the source row and correct only the copied experiment data or your working assumption.
Output differs	Reconcile IDs, row counts, denominators and units before changing the formula.
Chart does not open	Open the PNG directly from `output/`; the evidence CSV remains authoritative.

Note: The same detailed guide and checklist are also packaged in `labs/Lab-06-reliability-security-guardrails/` as Markdown and PDF.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 07 — Build a DORA and Problem-Management Evidence Dashboard

Field	Lab specification
Topic	Topic 07
Objective	Apply K5 and A6 by calculating documented delivery metrics and linking them to tracked problem-management actions.
Scenario	Leadership wants one slide of delivery-performance evidence, while engineering needs definitions precise enough to reproduce the numbers.
Output	A dora_metrics.csv, problem_actions.csv and dora_dashboard.png with metric definitions.
Tools	R 4.x · base R · DORA · problem records · reporting

Before you run the script

- Open a terminal in this lab's folder and confirm `Rscript --version`.
- Inspect every source CSV and note its row count and key identifiers.
- Preserve the supplied source data; make a copy for sensitivity experiments.
- Create no production connection and add no real personal, credential or customer information.

Step-by-step procedure

1. Review deployments.csv and problem_actions.csv; confirm each metric has the fields needed for reproducible calculation.

```
Rscript -e 'str(read.csv("data/deployments.csv"));
str(read.csv("data/problem_actions.csv"))'
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

2. Run dora_metrics.R to calculate throughput, instability and action-tracking measures.

```
Rscript dora_metrics.R
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

3. Inspect dora_metrics.csv and compare the current five-metric DORA model with the historical four-key vocabulary in the v3.4 guide.

```
Rscript -e 'print(read.csv("output/dora_metrics.csv"))'
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

4. Open dora_dashboard.png and identify one improvement hypothesis; do not rank individuals or unrelated services.

```
open output/dora_dashboard.png
```


© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

5. Review problem_actions.csv for overdue or unverified corrective actions.

```
Rscript -e 'print(read.csv("output/problem_actions.csv"))'
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

6. Record the metric definition, data source, owner and next review date in CHECKLIST.md.

```
open CHECKLIST.md
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

Evidence to retain

- output/dora_metrics.csv
- output/problem_actions.csv
- output/dora_dashboard.png
- completed CHECKLIST.md

Acceptance criteria

- Five current DORA metrics
- Definitions and denominators explicit
- Open actions tracked
- Improvement hypothesis avoids individual ranking

Final verification

Five DORA measures are present; the deployment counts reconcile; every open action has an owner and due date; dashboard values match the CSV.

Troubleshooting

Symptom	Check and recovery
Rscript not found	Confirm R is installed, reopen the terminal and verify PATH with `Rscript --version`.
Cannot open data file	Confirm the terminal is inside this Lab-XX folder and the `data/` directory retains its name.
Validation stop	Read the named rule, inspect the source row and correct only the copied experiment data or your working assumption.
Output differs	Reconcile IDs, row counts, denominators and units before changing the formula.
Chart does not open	Open the PNG directly from `output/`; the evidence CSV remains authoritative.

Note: The same detailed guide and checklist are also packaged in `labs/Lab-07-dora-problem-dashboard/` as Markdown and PDF.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Lab 08 — Capstone — Produce a Verified DevOps Problem Recommendation

Field	Lab specification
Topic	Topic 08
Objective	Apply A1–A6 in one connected case: manage the lifecycle, analyse root cause, prioritise, recommend controls and create monitorable documentation.
Scenario	Northstar Retail's checkout service has repeat latency failures after dependency changes. You are the cross-functional problem lead preparing a decision-ready problem record for leadership.
Output	A problem_record.md and recommendations.csv that can be audited from detection through verification.
Tools	R 4.x · base R · evidence synthesis · problem record

Before you run the script

- Open a terminal in this lab's folder and confirm `Rscript --version`.
- Inspect every source CSV and note its row count and key identifiers.
- Preserve the supplied source data; make a copy for sensitivity experiments.
- Create no production connection and add no real personal, credential or customer information.

Step-by-step procedure

1. Review all four capstone datasets and write an initial problem statement that separates impact from assumed cause.

```
Rscript -e 'for(f in list.files("data",full.names=TRUE)) {cat("\n",f,"\n"); print(read.csv(f))}'
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

2. Run capstone_problem_record.R to validate referential integrity and generate the first evidence synthesis.

```
Rscript capstone_problem_record.R
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

3. Open recommendations.csv and refine each action so it names control type, owner, due date, metric and verification threshold.

```
open output/recommendations.csv
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

4. Open problem_record.md and complete the root-cause and contributing-conditions narrative using only supported evidence.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

```
open output/problem_record.md
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

5. Use the acceptance checklist to test K/A traceability, lifecycle status and recurrence-prevention logic.

```
open CHECKLIST.md
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

6. Present a three-minute recommendation: problem, evidence, selected controls, trade-off and how success will be verified.

```
open output/problem_record.md
```

Verification checkpoint: compare the observed row count, value or file with the expectation in the next step; record any discrepancy before continuing.

Evidence to retain

- output/problem_record.md
- output/recommendations.csv
- completed CHECKLIST.md
- three-minute briefing

Acceptance criteria

- A1–A6 traceability complete
- Cause versus hypothesis distinguished
- At least three proportional controls
- Each action has owner, due date and verification threshold

Final verification

All referenced IDs reconcile; the record covers detection through learning; three recommendations include owner, due date and verification; unsupported causal claims are labelled as hypotheses.

Troubleshooting

Symptom	Check and recovery
Rscript not found	Confirm R is installed, reopen the terminal and verify PATH with `Rscript --version`.
Cannot open data file	Confirm the terminal is inside this Lab-XX folder and the `data/` directory retains its name.
Validation stop	Read the named rule, inspect the source row and correct only the copied experiment data or your working assumption.
Output differs	Reconcile IDs, row counts, denominators and units before changing the formula.
Chart does not open	Open the PNG directly from `output/`; the evidence CSV remains authoritative.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Note: The same detailed guide and checklist are also packaged in `labs/Lab-08-capstone-problem-record/` as Markdown and PDF.

Assessment Readiness

Instrument	Count	Duration	Coverage	Evidence source
WA-SAQ	5 open-ended questions	60 minutes	K1–K5	Topics 01–08 and concept explanations
Case Study	3 connected tasks	60 minutes	A1–A6	Labs 01–08, especially Lab 08 capstone

- Question 1: explain lifecycle principles and shared DevOps ownership (K1).
- Question 2: explain tools, processes and automated detection choices (K2).
- Question 3: explain investigation and diagnosis using evidence (K3).
- Question 4: explain transparent severity, frequency, impact and effort prioritisation (K4).
- Question 5: explain documentation, metrics and verification practices (K5).
- Case Study Task 1: lifecycle and automated detection (A1, A2).
- Case Study Task 2: root-cause investigation and prioritisation (A3, A4).
- Case Study Task 3: recommendations, recurrence prevention and tracked verification (A5, A6).

Note: Use authorised official study materials only. The PeopleCert exam is a separate certification examination; the WSQ course assessment remains open-ended WA plus Case Study as approved in the Course Proposal.

Glossary

Term	Meaning
AIOps	Analytics and machine learning used to assist operational detection, correlation and diagnosis.
Blameless review	Inquiry that examines decisions and conditions to improve the system rather than find a scapegoat.
CALMS	Culture, Automation, Lean, Measurement and Sharing transformation lens.
Canary deployment	Progressive release to a small audience before wider exposure.
Change fail rate	Share of deployments requiring immediate intervention after production change.
Change lead time	Time from committed change to successful production deployment.
ChatOps	Operational collaboration and automation performed through a shared chat context.
Continuous delivery	Capability to keep every accepted change deployable on demand.
Continuous deployment	Automatic production deployment of every accepted change.
Continuous integration	Frequent mainline integration validated by automated build and tests.
Cultural debt	Accumulated behaviours and workarounds that make future collaboration and change harder.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Deployment frequency	How often an application or service is deployed.
Deployment rework rate	Share of deployments that are unplanned work caused by production incidents.
DevSecOps	Integration of security responsibility and automated evidence throughout delivery and operation.
Error budget	Allowed unreliability implied by an SLO, used to balance reliability and change.
Failed deployment recovery time	Time to restore service after a deployment causes impairment.
Flow efficiency	Active work time divided by total elapsed lead time.
Infrastructure as Code	Versioned, reviewable machine-readable infrastructure definition.
Kanban	Visual pull system with explicit policies and limits on work in progress.
Observability	Ability to understand internal system state from outputs such as logs, metrics and traces.
Platform engineering	Product approach to safe self-service paths that reduce developer cognitive load.
Problem	Cause, or potential cause, of one or more incidents.
Problem record	Controlled documentation of identity, impact, evidence, cause, action, ownership and verification.
Psychological safety	Shared belief that speaking up, asking for help and reporting mistakes will not invite punishment.
SRE	Engineering approach to reliable operation using SLOs, error budgets, automation and learning.
Theory of Constraints	Method for improving a system by identifying and managing its current constraint.
Three Ways	Flow, feedback, and continual experimentation and learning.
Value stream	End-to-end activities from customer request to realised value.
Westrum model	Typology of pathological, bureaucratic and generative information cultures.
Work in progress	Started work not yet finished; excessive WIP increases queueing and lead time.

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg